॥श्रीगणेशाय नमः॥

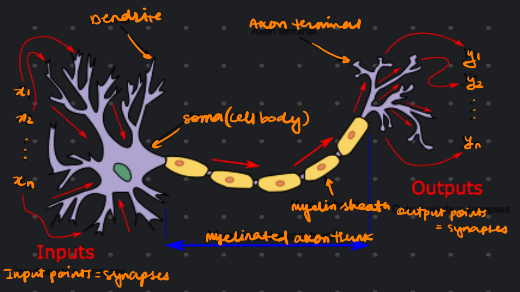
Neural Networks

Artificial neural networks

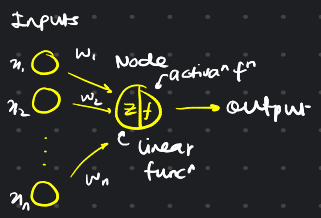
- ANNs are computational models inspired by the human brain.
- They consist of neurons (nodes) arranged in layers, with connections (weights) that process and transform inputs to outputs.
- ANNs solve complex tasks like image recognition, speech processing, and natural language understanding.
- ANNs handle non-linear relationships better than traditional algorithms.

ANNs → composed of neurons → Input layer, hidden layers, output layer

Neuron (neural unit)



Biological neuron



Artificial neuron

- Each artificial neuron computes $z = \sum_{i=1}^{n} w_i x_i + b$

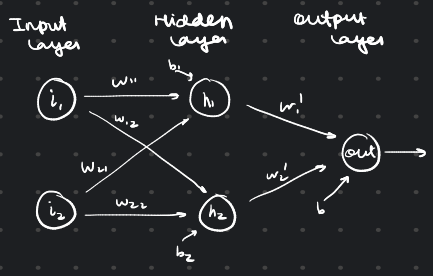
output = $f(z)$ ← activation function

- ANNS:
  - Feedforward neural networks
  - Convolutional neural networks
  - Recurrent neural networks
  - Modular neural networks

Activation functions

- An activation function is a mathematical function applied to the output of a neuron.
- Introduces non-linearity in the neural net and allows the neural network to learn and represent complex patterns.
- It is used to bound the outputs to specific ranges such as [0,1] or [-1, 1].
- They help us enable gradient-based optimizations: differentiable activation functions allow gradient descent to adjust weights effectively.

Why activation functions?



Output of $h_1 = w_{11} i_1 + w_{21} i_2 + b_1 = z_1$

Output of $h_2 = w_{12} i_1 + w_{22} i_2 + b_2 = z_2$

output = $w_1' z_1 + w_2' z_2 + b$

$= i_1 (w_1' w_{11} + w_2' w_{12}) + i_2 (w_1' w_{21} + w_2' w_{22}) + (w_1' b_1 + w_2' b_2 + b)$

$= \hat{w_1} i_1 + \hat{w_2} i_2 + \hat{b}$

= linear combination of inputs

- It doesn't matter how many hidden layers or how many neurons we have in every layer in a neural network, the final output will always be a linear combination of the inputs if we do NOT add an activation function to the outputs of a neuron.
- The activation function does non-linear transformations to the input, making it capable to learn and perform more complex tasks.

Different activation functions

Linear activation function

$f(z) = z$

$f'(z) = 1$

Range: $(-\infty, \infty)$



- Cannot capture complex patterns due to lack of non-linearity.
- Rarely used in practice.

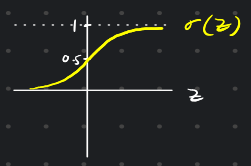
Sigmoid activation function

$$f(z) = \sigma(z) = \frac{1}{1+e^{-z}}$$

$$f'(z) = -(1+e^{-z})^{-2} \cdot e^{-z} \cdot (-1) = \frac{e^{-z}}{(1+e^{-z})^2} = \frac{1}{1+e^{-z}} \cdot \frac{e^{-z}}{1+e^{-z}} = f(z)\left(\frac{1+e^{-z}-1}{1+e^{-z}}\right) = f(z)(1-f(z))$$

$f'(z) = f(z) \cdot (1 - f(z))$

range: $(0, 1)$

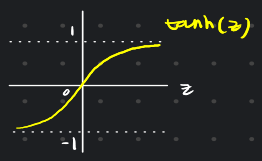


- Problem:
  Vanishing gradients: Derivatives become very small for large values of $|z|$, slowing down the learning.
- Advantage:
  - Maps inputs to a probability-like range.
  - Used in binary classification tasks.

tanh activation function (hyperbolic tangent)

$$f(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

range: $(-1, 1)$



$\tanh = 2(\sigma(2z) - 1)$

$$f'(z) = \frac{(e^z + e^{-z})(e^z + e^{-z}) - (e^z - e^{-z})(e^z - e^{-z})}{(e^z + e^{-z})^2} = 1 - (f(z))^2$$

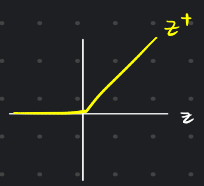
$f'(z) = 1 - (f(z))^2$

Advantages:
- Zero-centred output helps reduce bias shifts during weight updates.
- Better for hidden layers as compared to sigmoid.

Disadvantage:
- Vanishing gradient problem.

ReLU (rectified linear unit)

$f(z) = \max(0, z) = z^+ = \frac{z + |z|}{2}$



we consider $f'(z) = 0$ at $z = 0$.

$$f'(z) = \begin{cases} 0 & z \le 0 \\ 1 & z > 0 \end{cases}$$

range: $[0, \infty)$

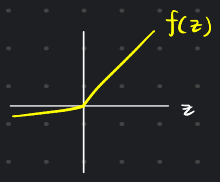
Advantages:
- Computationally efficient.
- Alleviates the vanishing gradient problem for $z > 0$.

Disadvantages:
- Dying ReLU problem: Neurons will become inactive if $z \le 0$ (the neuron dies (╥﹏╥))

Leaky ReLU (parametric ReLU)

$$f(z) = \begin{cases} \alpha z & \text{if } z \le 0 \\ z & \text{if } z > 0 \end{cases}$$

α is a small +ve constant.
e.g. $\alpha = 0.01$



$$f'(z) = \begin{cases} \alpha & \text{if } z \le 0 \\ 1 & \text{if } z > 0 \end{cases}$$

assume $z = 0$

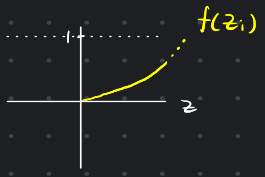
range : $(-\infty, \infty)$

Advantages:
- Avoids the dying ReLU problem.
- Performs well.

Softmax function

- Used in the output layer in multiclass classification tasks.
- Used to convert logits into probability values.

$$f(z_i) = \frac{e^{z_i}}{\sum_i e^{z_i}}$$



range : $(0, 1)$

Loss functions

- A loss function quantifies the difference between the predicted output of a NN and the true value (ground truth).
- It serves as the feedback mechanism for the model to improve the accuracy of predictions during training.

Key properties of loss functions

- Continuity: Smooth to enable gradient computations.
- Differentiability: Differentiable to enable gradient computations. ← ReLU is assumed to give derivative 0 at $z=0$.
- Convexity: A convex loss function ensures convergence to a global minima in simpler models.
- Scale sensitivity: Should align with the problem's scale. E.g. cross-entropy loss for probabilities in classification tasks.



Loss functions for regression

mean square error: $MSE = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$

$y = [3, 5, 2]$, $\hat{y} = [2.5, 5.2, 1.8]$ $\Rightarrow MSE = \frac{1}{3}(0.5^2 + 0.2^2 + 0.2^2) = 0.11$

→ used for continuous outputs (regression).
→ penalizes large errors more than small errors (quadratic growth).
→ convex and differentiable.

Mean Absolute Error: $MAE = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i|$

$y = [3, 5, 2]$, $\hat{y} = [2.5, 5.2, 1.8]$ $MAE = \frac{1}{3}(0.5 + 0.2 + 0.2) = 0.3$

→ Penalizes the error linearly.
→ less sensitive to outliers than MSE.
→ used for continuous outputs.

used despite being non-differentiable at $y_i = \hat{y}_i$

$$\frac{\partial MAE}{\partial \hat{y}} = \begin{cases} +1, & y_i > \hat{y}_i \\ -1, & y_i < \hat{y}_i \\ ?, & y_i = \hat{y}_i \end{cases}$$

the function is non-differentiable which is already a minima for that points hence derivative is not reqd at this point

Huber loss:

$$L_\delta(y, \hat{y}) = \begin{cases} \frac{1}{2}(y - \hat{y})^2 & \text{if } |y - \hat{y}| \le \delta \\ \delta|y - \hat{y}| - \frac{1}{2}\delta^2 & \text{if } |y - \hat{y}| > \delta \end{cases}$$

} not too far away, penalize more
} diff. very large

→ combines benefits of MSE and MAE.
→ hyperparameter δ

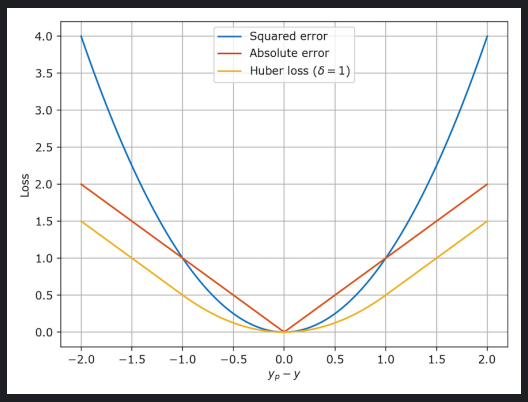
$y = [3, 5, 2]$, $\hat{y} = [2.5, 5.2, 1.8]$ $\delta = 0.3$

$|3 - 2.5| = 0.5$ $|5 - 5.2| = 0.2$ $|2 - 1.8| = 0.2$

$\left[(0.3)(0.5) - \frac{1}{2}(0.3)^2\right]$ $\frac{1}{2}(0.2^2)$ $\frac{1}{2}(0.2^2)$

Total = 0.145

Huber loss = $\frac{0.145}{3} = 0.04833$



Loss functions for classification

Binary cross-entropy loss

$$L = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)\right]$$

y_i: actual label (0/1); $\hat{y}_i$: predicted prob $\in [0,1]$

$y = [1.0, 1, 1, 0]$ $\quad \hat{y} = [0.9, 0.1, 0.8, 0.7, 0.2]$

y	$\hat{y}$	$1-y$	$1-\hat{y}$	$\ln(\hat{y})$	$\ln(1-\hat{y})$
1	0.9	0	0.1	-0.1053	-2.3025
0	0.1	1	0.9	-2.3025	-0.1053
1	0.8	0	0.2	-0.2231	-1.6094
1	0.7	0	0.3	-0.3567	-1.2039
0	0.2	1	0.8	-1.6094	-0.2231

$$L = -\frac{1}{5}[-0.1053 - 0.1053 - 0.2231 - 0.3567 - 0.2231)$$

$$= \frac{1}{5}($$

$$= 0.203$$

→ used in binary classificn task.
→ $\hat{y}$ is generally prob. value calculated by sigmoid

Categorical cross-entropy loss:

$$L = -\frac{1}{N}\sum_{i=1}^{N}\sum_{j=1}^{C} y_{ij}\log(\hat{y}_{ij})$$

y_{ij}: actual label (1 if class for label i is j); $\hat{y}_{ij}$: prob. of jth class for i^{th} example.

Classes A, B, C. Actual for 3 examples: [A, B, C] Predicted prob.

Data point	A	B	C
1	0.8	0.1	0.1
2	0.2	0.7	0.1
3	0.1	0.2	0.7

$$CCE = -\frac{1}{3}\left[\ln 0.8 + \ln 0.7 + \ln 0.7\right] = 0.3122$$

Hinge loss:

$$L = \frac{1}{N}\sum_{i=1}^{N}\max(0, 1 - y_i \cdot \hat{y}_i)$$

y_i: -1/1; $\hat{y}_i$: predicted logits (not prob.; raw output)

$y = [1, -1, 1, 1, -1]$ $\quad \hat{y} = [0.8, -0.9, -0.3, 0.7, -0.3]$

$$HL = \frac{1}{5}\left[\max(0, 0.2), \max(0, 0.1), \max(0, 1.3), \max(0, 0.3), \max(0, 0.7)\right] = \frac{1}{5}(0.2 + 0.1 + 1.3 + 0.3 + 0.7)$$

$$= 0.52$$

Custom loss function

- For advanced tasks, custom loss functions can be designed for specific objectives like when we have class imbalance or multi-objective optimization.

Eg α(MSE) + β(Huber loss)

In class imbalance, we could penalize the model more for getting the minority class wrong so that it learns the minority as well.

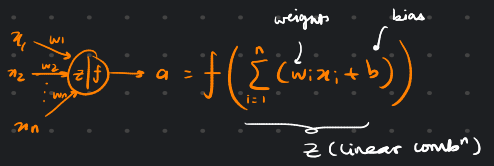
class 1 → 100 examples
class 2 → 10 examples

$$\frac{1}{11} y_i \ln \hat{y}_i + \frac{10}{11}(1-\hat{y}_i)\ln(1-\hat{y}_i)$$

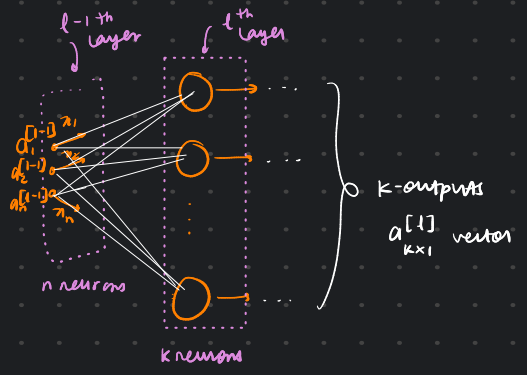
Feedforward neural networks

- A FFNN is the simplest ANN where the information flows in one direction, from inputs to outputs.
- Unlike in RNNs, there are no cycles or feedback loops.
- Unidirectional flow: Data flows from the input layer to the output layer without looping back.
- Layered structure:
  - Input layer:
    - Accepts the raw data features.
      vector x = [x_1 ... x_n]
  - Hidden layer(s):
    - Perform computations and extract meaningful features.
      (weighted summation + activation function).
  - Output layer:
    - Produces the final prediction.
      (class label/probabilities/continuous value)
- Fully connected: Every neuron in one layer is connected to every neuron in the next.

For a single neuron



For an entire layer (l):



$$z^{[l]}_{k\times 1} = w^{[l]}_{k\times n} \cdot a^{[l-1]}_{n\times 1} + b^{[l]}_{k\times 1}$$

$$\begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_k \end{bmatrix}^{[l]}_{k\times 1} = \begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1n} \\ w_{21} & w_{22} & \cdots & w_{2n} \\ \vdots & & & \vdots \\ w_{k1} & w_{k2} & \cdots & w_{kn} \end{bmatrix}_{k\times n} \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{bmatrix}^{(l-1)}_{n\times 1} + \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_k \end{bmatrix}^{(l)}_{k\times 1}$$

$z_{i\ k\times 1}^{[l]}$ = ith row * prev outputs + b_i

$= (w_{i1}a_1 + w_{i2}a_2 + \cdots + w_{in}a_n) + b_i$

$a_{i\ k\times 1}^{[l]} = f(z_i^{[l]})$

of weights for ith neuron = $n_{prev} + 1$ (bias)

of weights for entire layer = $(n_{prev} + 1)\, n_{curr}$

(n_{prev}: neurons in prev layer; n_{curr}: neurons in curr layer)

Multi-layer perceptron

These are special types of ANNs in which:
- there's atleast 1 hidden layer (multi).
- activation functions are present at every layer to introduce non-linearity to make the network capable of solving complex problems.

Architecture:

L layers : 1 + (L-2) + 1
(Input, Hidden, output)

NOTE: A NN with a single hidden layer with ReLU activation is a universal approximator.

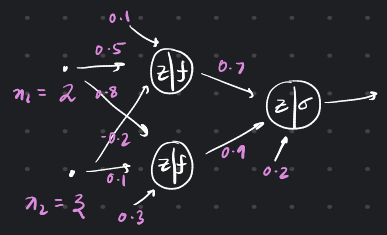
e.g. Input layer + Hidden layer + Output layer
(2 inputs) (2 neurons) (1 neuron)
(activation = ReLU) (activation = σ)

For input $\begin{bmatrix} x_1 = 2 \\ x_2 = 3 \end{bmatrix}$ calculate the final output.

$$W^{[1]} = \begin{bmatrix} 0.5 & -0.2 \\ 0.8 & 0.1 \end{bmatrix} \quad W^{[2]} = \begin{bmatrix} 0.7 & 0.9 \end{bmatrix}$$

$$b^{[1]} = \begin{bmatrix} 0.1 \\ -0.3 \end{bmatrix} \quad b^{[2]} = \begin{bmatrix} 0.2 \end{bmatrix}$$

→

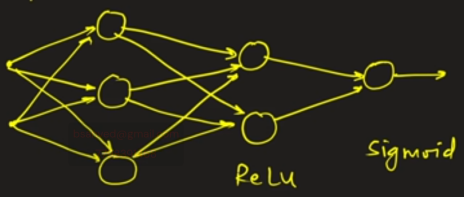


$$\begin{bmatrix} z_1 \\ z_2 \end{bmatrix}^{[1]} = \begin{bmatrix} 0.5 & -0.2 \\ 0.8 & 0.1 \end{bmatrix} \begin{bmatrix} 2 \\ 3 \end{bmatrix} + \begin{bmatrix} 0.1 \\ -0.3 \end{bmatrix} = \begin{bmatrix} 0.5 \\ 1.6 \end{bmatrix}$$

$$\begin{bmatrix} a_1 \\ a_2 \end{bmatrix}^{[1]} = \text{ReLU} \begin{bmatrix} 0.5 \\ 1.6 \end{bmatrix} = \begin{bmatrix} 0.5 \\ 1.6 \end{bmatrix}$$

$$[z_1]^{[2]} = \begin{bmatrix} 0.7 & 0.9 \end{bmatrix} \begin{bmatrix} 0.5 \\ 1.6 \end{bmatrix} + [0.2] = 1.99$$

$$[a_1]^{[2]} = [\sigma(1.99)] = \left[\frac{1}{1+e^{-1.99}}\right] = [0.8797]$$



$$W^{[1]} = \begin{bmatrix} 0.5 & -0.3 \\ 0.8 & 0.1 \\ 0.2 & 0.4 \end{bmatrix} \quad b^{[1]} = \begin{bmatrix} 0.1 \\ 0.2 \\ 0.3 \end{bmatrix}$$

$$W^{[2]} = \begin{bmatrix} -0.8 & 0.2 & 0.1 \\ 0.3 & -0.5 & 0.2 \end{bmatrix} \quad b^{[2]} = \begin{bmatrix} 0.7 \\ -0.8 \end{bmatrix}$$

$$W^{[3]} = [0.7 \ 0.3] \quad b^{[3]} = [-0.5]$$

Calculate the predicted output for $\begin{bmatrix} 2 \\ 3 \end{bmatrix}$ input.

$$z^{[1]} = \begin{bmatrix} 0.5 & -0.3 \\ 0.8 & 0.1 \\ 0.2 & 0.4 \end{bmatrix} \begin{bmatrix} 2 \\ 3 \end{bmatrix} + \begin{bmatrix} 0.1 \\ 0.2 \\ 0.3 \end{bmatrix} = \begin{bmatrix} 0.1 \\ 1.9 \\ 1.6 \end{bmatrix} + \begin{bmatrix} 0.1 \\ 0.2 \\ 0.3 \end{bmatrix} = \begin{bmatrix} 0.2 \\ 2.1 \\ 1.9 \end{bmatrix}$$

$$a^{[1]} = \begin{bmatrix} 0.2 \\ 2.1 \\ 1.9 \end{bmatrix}$$

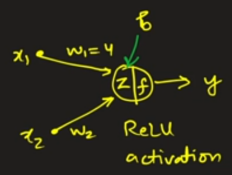
$$z^{[2]} = \begin{bmatrix} -0.8 & 0.2 & 0.1 \\ 0.3 & -0.5 & 0.2 \end{bmatrix} \begin{bmatrix} 0.2 \\ 2.1 \\ 1.9 \end{bmatrix} + \begin{bmatrix} 0.7 \\ -0.8 \end{bmatrix} = \begin{bmatrix} 0.45 \\ -0.61 \end{bmatrix} + \begin{bmatrix} 0.7 \\ -0.8 \end{bmatrix} = \begin{bmatrix} 1.15 \\ -1.41 \end{bmatrix}$$

$$a^{[2]} = \begin{bmatrix} 1.15 \\ 0 \end{bmatrix}$$

$$z^{[3]} = [0.7 \ 0.3] \begin{bmatrix} 1.15 \\ 0 \end{bmatrix} - [0.5] = [0.805] - [0.5] = [0.305]$$

$$a^{[3]} = [\sigma(0.305)] = 0.5757$$

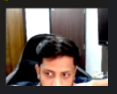
Q: Single output layer neural network is given :-



Dataset :

x_1	x_2	y
1	2	3
2	-2	0

The weights of this neural network are trained to fit the dataset given. Calculate the largest integral value of b which fits the given dataset.



→ $3 = 4(1) + 2(w_2) + b$

$2w_2 + b = -1$

$0 = 4(2) + (-2)(w_2) + b$

$b - 2w_2 = -8$

$2w_2 - b = 8$

$4w_2 = 7$

$w_2 = \frac{7}{4}$

$\Rightarrow \frac{7}{2} + b = -1$

$b = -1 - \frac{7}{2} = -\frac{9}{2}$

$\text{ReLU}(4(1) + 2w_2 + b) = 3$

$4 + 2w_2 + b = 3$

$w_2 = \frac{-1 - b}{2}$

$\text{ReLU}(8 - 2w_2 + b) = 0$

$8 - 2w_2 + b \leq 0$

$8 - (-1 - b) + b \leq 0$

$8 + 1 + b + b \leq 0$

$2b \leq -9$

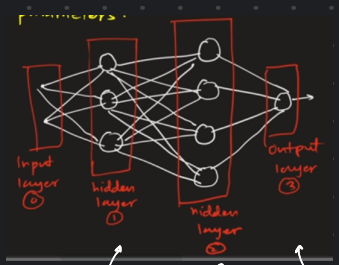
$b \leq -\frac{9}{2}$

$b \leq -4.5$

Parameters in MLP

- In a MLP, parameters refer to the components of the model that are learned during training to minimize the loss function.
- These parameters are
    1. Weights (W):
        - Represent the strength of connections between neurons in adjacent layers.
        - Each connection between 2 neurons has an associated weight.
    2. Biases (b):
        - Added to the weighted sum of inputs to introduce flexibility in learning.
        - Each neuron has its own bias.
        - Allow the model to shift the activation function, enabling it to learn more complex patterns.

Eg: For an MLP with 2 input neurons, 3 hidden neurons in the first layer, 4 hidden neurons in the second layer and 1 output neuron, Calculate the total number of parameters.



of parameters
= 2·3+3
= 9

3·4+4
= 16

4·1+1=5

$(n_{prev}+1)\,n_{curr}$

Q.43	Consider a Multi-Layer Perceptron (MLP) model with one hidden layer and one output layer. The hidden layer has 10 neurons, and the output layer has 3 neurons. The input to the MLP is a 5-dimensional vector. Each neuron is connected to every neuron in the previous layer, and a bias term is included for each neuron. The activation function used is the sigmoid function. Calculate the total number of trainable parameters in this MLP model.

$(5+1)\,10 + (10+1)\,3 = 60 + 30 + 3 = 93$

Backpropogation

Training a model

- Training a neural network involves adjusting its parameters (weights and biases) to minimize the error (loss) between its predictions and the ground truth. This is achieved through an iterative process.

Steps

1. Weight initialization: Assign random or specific initial values to weights and biases.
2. Forward propogation: Compute outputs from the input layer to the output layer.
3. Loss computation: Measure the error between the predicted and true outputs using a loss function.
4. Backpropogation: Compute the gradients of the loss w.r.t. weights and biases.
5. Weight update: Adjust weights and biases using an optimization algorithm.
6. Repeat: Iterate over the training data until model convergence or a specified # of epochs is reached.

Forward propogation:
- Passing the input data through the network to compute the output.

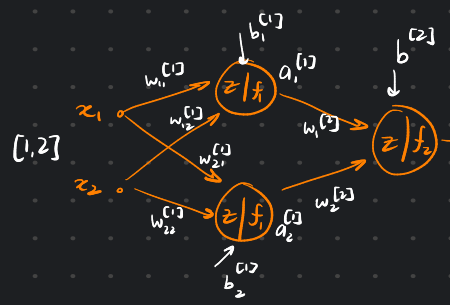
Loss computation:
- Calculate the error between the predicted and the actual values.

$L = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$ For regression (MSE)

$L = -\frac{1}{N}\sum_{i=1}^{N}\left(y_i \ln \hat{y}_i + (1-y_i)\ln(1-\hat{y}_i)\right)$ For binary classificn (BCE)

Backpropogation

- Computes the gradients of the loss function w.r.t. weights and biases using the chain rule.



Loss function: BCE
f_1: ReLU
f_2: sigmoid

$\hat{y} = 0.7$
$y = 1$

$L = -[1 \cdot \ln(0.7)] \approx 0.357$

There are 9 weights.

We can write the loss funcn in terms of weights.

$L = -[y \ln \hat{y} + (1-y)\ln \hat{y}]$

$\frac{\partial L}{\partial \hat{y}} = -\left[\frac{y}{\hat{y}} - \frac{(1-y)}{(1-\hat{y})}\right] = \frac{(1-y)}{(1-\hat{y})} - \frac{y}{\hat{y}}$ $\hat{y}$ is funcn of $z^{[2]}$

$\hat{y} = a^{[2]} = \sigma(z^{[2]})$ ← ben sigmoid

$\frac{\partial \hat{y}}{\partial z^{[2]}} = \sigma(z^{[2]})(1 - \sigma(z^{[2]})) = a^{[2]}(1 - a^{[2]}) = \hat{y}(1-\hat{y})$

$z^{[2]}$ is a funcn of weights.

$z^{[2]} = w_1^{[2]} \cdot a_1^{[1]} + w_2^{[2]} \cdot a_2^{[1]} + b^{[2]}$

$\frac{\partial z^{[2]}}{\partial w_1^{[2]}} = a_1^{[1]}$ ← all outputs are already obtained from forward prop.

Say $a_1^{[1]} = 0.6$

Using the chain rule,

$\frac{\partial L}{\partial w_1^{[2]}} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z^{[2]}} \cdot \frac{\partial z^{[2]}}{\partial w_1^{[2]}}$ → backpropogn

$= \left(\frac{-1}{0.7}\right) \cdot (0.7)(1-0.7) \cdot (0.6) = -(0.3)(0.6) = -0.18$

derivative is -ve so we move $w_1^{[2]}$ in the +ve direction.

Weight update

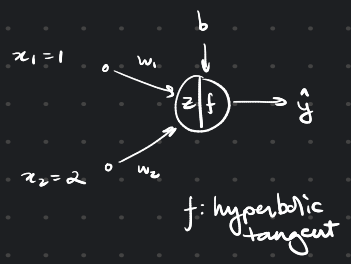
- Weights are updated using an optimization techniques like gradient descent.

$$w_{(i+1)} = w_{(i)} - \eta \cdot \frac{\partial L}{\partial w_{(i)}}$$

$\frac{\partial L}{\partial w_1^{[2]}} = -0.18$ $\qquad w_{1\,new}^{[2]} = w_{1\,old}^{[2]} - \eta \cdot \frac{\partial L}{\partial w_{1\,old}^{[2]}}$

$= 0.3 - (0.1)(-0.18) = 0.318$

Q.



f: hyperbolic tangent

for data point (1,2), the actual output is 1.
calculate the weights and bias after 1 iteration of GD.
$w_1 = 0.5$, $w_2 = -0.4$, $b = 0.2$ } initial values.
$\eta = 0.1$
use MSE as loss func^n.

→ 1st forward pass :

$z = x_1 w_1 + x_2 w_2 + b = (1)(0.5) + (2)(-0.4) + (0.2) = -0.1$

$\hat{y} = f(z) = \tanh(-0.1) = -0.0997$

$\text{Loss} = \frac{1}{1}\sum (y - \hat{y})^2 = \frac{1}{1}[(1 + 0.0997)^2] = 1.2093$

Back prop :

$L = (y - \hat{y})^2 = y^2 - 2y\hat{y} + \hat{y}^2$

$\frac{\partial L}{\partial \hat{y}} = -2y + 2\hat{y} = -2(1) + 2(-0.0997) = -2.1994$

$\hat{y} = \tanh(z)$

$\frac{\partial \hat{y}}{\partial z} = 1 - (\tanh(z))^2 = 1 - (\hat{y})^2 = 1 - (-0.0997)^2 = 0.99$

$\frac{\partial z}{\partial w_1} = x_1 = 1$ $\qquad \frac{\partial z}{\partial w_2} = x_2 = 2$ $\qquad \frac{\partial z}{\partial b} = 1$

$\delta = \frac{\partial L}{\partial z} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} = (-2.1994)(0.99) = -2.17774$

$\frac{\partial L}{\partial w_1} = \delta \cdot \frac{\partial z}{\partial w_1} = -2.17774$ $\qquad \frac{\partial L}{\partial w_2} = (-2.17774)(2) = -4.35548$ $\qquad \frac{\partial L}{\partial b} = (-2.17774)(1) = -2.17774$

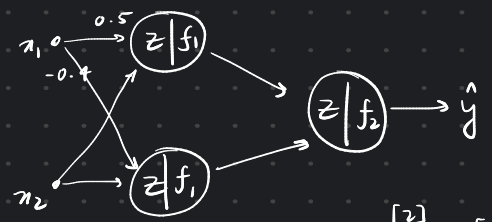
$w_1^{new} = w_1 - \eta \frac{\partial L}{\partial w_1}$ $\qquad w_2^{new} = w_2 - \eta \frac{\partial L}{\partial w_2}$ $\qquad b^{new} = b - \eta \frac{\partial L}{\partial b}$

$= (0.5) - (0.1)(-2.17774)$ $\qquad = (-0.4) - (0.1)(-4.35548)$ $\qquad = (0.2) - (0.1)(-2.17774)$

$= 0.71774$ $\qquad = 0.035548$ $\qquad = 0.417774$

$\hat{y}_{new} =$

Q. For binary classificⁿ:



f_1: ReLU

f_2: σ

Loss: Binary cross-entropy

For $x=(1,2)$, the actual output is $y=1$.

Calculate the updated weights after 1 iterⁿ of GD.

$$w^{[1]} = \begin{bmatrix} 0.5 & -0.4 \\ 0.3 & 0.7 \end{bmatrix} \qquad w^{[2]} = [0.6 \quad 0.8]$$

$$b^{[1]} = \begin{bmatrix} 0.2 \\ -0.1 \end{bmatrix} \qquad b^{[2]} = [-0.5]$$

→ Forward prop:

$$z^{[1]} = \begin{bmatrix} 0.5 & -0.4 \\ 0.3 & 0.7 \end{bmatrix}\begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 0.2 \\ -0.1 \end{bmatrix} = \begin{bmatrix} -0.1 \\ 1.6 \end{bmatrix} \qquad z^{[1]} = w^{[1]} a^{[0]} + b^{[1]}$$

$$a^{[1]} = \text{ReLU}\begin{bmatrix} -0.1 \\ 1.6 \end{bmatrix} = \begin{bmatrix} 0 \\ 1.6 \end{bmatrix}$$

$$z^{[2]} = [0.6 \quad 0.8]\begin{bmatrix} 0 \\ 1.6 \end{bmatrix} + [-0.5] = [0.78] \qquad z^{[2]} = w^{[2]} \cdot a^{[1]} + b^{[2]}$$

$$a^{[2]} = \sigma([0.78]) = [0.686]$$

$$\text{Loss } L = -[y \ln \hat{y} + (1-y)\ln(1-\hat{y})] = -\ln(0.686) = 0.377$$

Backward prop:

$$\frac{\partial L}{\partial \hat{y}} = -\frac{\partial}{\partial \hat{y}}[y \ln \hat{y} + (1-y)\ln(1-\hat{y})] = -\left[\frac{y}{\hat{y}} - \frac{(1-y)}{(1-\hat{y})}\right] = -\frac{1}{0.686} = -1.46$$

$$\frac{\partial \hat{y}}{\partial z^{[2]}} = \frac{\partial a^{[2]}}{\partial z^{[2]}} = a^{[2]}(1-a^{[2]}) = 0.686(1-0.686) = 0.215$$

$$\frac{\partial z^{[2]}}{\partial w^{[2]}} = a^{[1]} \qquad \frac{\partial z^{[2]}}{\partial b^{[2]}} = 1$$

updated wts:

$$w^{[2]}_{new} = w^{[2]} - \eta \frac{\partial L}{\partial w^{[2]}}$$

$$= [0.6 \quad 0.8] - (0.1)\underbrace{\left(\frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z^{[2]}}\right)}_{0.314}\left(\frac{\partial z^{[2]}}{\partial w^{[2]}}\right)$$

$$= [0.6 \quad 0.8] - (0.1)(0.314)[0 \quad 1.6]$$

$$= [0.6 \quad 0.8] - [0 \quad -0.05]$$

$$= [0.6 \quad 0.85]$$

$$b^{[2]}_{new} = b^{[2]} - \eta \frac{\partial L}{\partial b^{[2]}}$$

$$= [-0.5] - 0.0314[1]$$

$$= [-0.469]$$

For the 1st layer:

$$\frac{\partial z^{[2]}}{\partial a^{[1]}} = w^{[2]} \qquad \frac{\partial a^{[1]}}{\partial z^{[1]}} = \begin{cases} 0 & z^{[1]} \le 0 \\ 1 & z^{[1]} > 0 \end{cases}$$

$$\frac{\partial z^{[1]}}{\partial w^{[1]}} = a^{[0]} \qquad \frac{\partial z^{[1]}}{\partial b^{[1]}} = 1$$

$$\delta_2 = \frac{\partial L}{\partial z^{[1]}} = \frac{\partial L}{\partial z^{[2]}} \cdot \frac{\partial z^{[2]}}{\partial a^{[1]}} \cdot \frac{\partial a^{[1]}}{\partial z^{[1]}} = (-0.314)\begin{bmatrix} 0.6 \\ 0.8 \end{bmatrix} \odot \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ -0.251 \end{bmatrix}$$

element-wise multiplicⁿ

$z_1^{[1]} \le 0$

$z_2^{[1]} > 0$

$$\frac{\partial L}{\partial W^{[1]}} = \begin{bmatrix} 0 \\ -0.251 \end{bmatrix} \begin{bmatrix} 1 & 2 \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ -0.251 & -0.502 \end{bmatrix}$$

$$\frac{\partial L}{\partial b^{[1]}} = \begin{bmatrix} 0 \\ -0.251 \end{bmatrix} \odot \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ -0.251 \end{bmatrix}$$

dying ReLU

Updated weights:

$$W^{[1]}_{new} = W^{[1]}_{old} - \eta \frac{\partial L}{\partial W^{[1]}} = \begin{bmatrix} 0.5 & -0.4 \\ 0.3 & 0.7 \end{bmatrix} - (0.1) \begin{bmatrix} 0 & 0 \\ -0.251 & -0.502 \end{bmatrix} = \begin{bmatrix} 0.5 & -0.4 \\ 0.325 & 0.75 \end{bmatrix}$$

$$b^{[1]}_{new} = b^{[1]} - \eta \frac{\partial L}{\partial b^{[1]}} = \begin{bmatrix} 0.2 \\ -0.1 \end{bmatrix} - (0.1) \begin{bmatrix} 0 \\ -0.251 \end{bmatrix} = \begin{bmatrix} 0.2 \\ 0.125 \end{bmatrix}$$

$$\frac{\partial L}{\partial w_{11}^{[1]}} = \underbrace{\frac{\partial L}{\partial a^{[2]}} \cdot \frac{\partial a^{[2]}}{\partial z^{[2]}}}_{-0.314} \cdot \underbrace{\frac{\partial z^{[2]}}{\partial a^{[1]}}}_{0.6} \cdot \underbrace{\frac{\partial a^{[1]}}{\partial z_1^{[1]}}}_{0} \cdot \frac{\partial z_1^{[1]}}{\partial w_{11}^{[1]}}$$

0 as $z_1^{[1]} < 0$

$$= 0$$

$$\frac{\partial L}{\partial w_{12}^{[1]}} = \underbrace{\frac{\partial L}{\partial a^{[2]}} \cdot \frac{\partial a^{[2]}}{\partial z^{[2]}}}_{0.314} \cdot \underbrace{\frac{\partial z^{[2]}}{\partial a^{[1]}}}_{0.8} \cdot \underbrace{\frac{\partial a^{[1]}}{\partial z_1^{[1]}}}_{1} \cdot \frac{\partial z_1^{[1]}}{}$$

Q.43 Consider the two neural networks (NNs) shown in Figures 1 and 2, with $ReLU$ activation ($ReLU(z) = \max\{0, z\}, \forall z \in \mathbb{R}$). $\mathbb{R}$ denotes the set of real numbers. The connections and their corresponding weights are shown in the Figures. The biases at every neuron are set to 0. For what values of p, q, r in Figure 2 are the two NNs equivalent, when x_1, x_2, x_3 are positive?

Figure 1

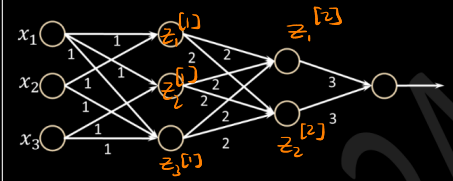
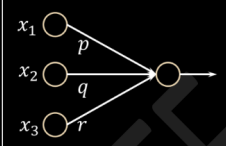


Figure 2



(A)	$p = 36, q = 24, r = 24$ ✓
(B)	$p = 24, q = 24, r = 36$
(C)	$p = 18, q = 36, r = 24$
(D)	$p = 36, q = 36, r = 36$

For NN1:

$z_1^{[1]} = x_1 + x_2$ $\quad a_1^{[1]} = x_1 + x_2$

$z_2^{[1]} = x_1 + x_3$ $\quad a_2^{[1]} = x_1 + x_3$

$z_3^{[1]} = x_1 + x_2 + x_3$ $\quad a_3^{[1]} = x_1 + x_2 + x_3$

$z_1^{[2]} = a_1^{[2]} = 2a_1^{[1]} + 2a_2^{[1]} + 2a_3^{[1]} = 6x_1 + 4x_2 + 4x_3$

$a_2^{[2]} = 2a_1^{[1]} + 2a_2^{[1]} + 2a_3^{[1]} = 6x_1 + 4x_2 + 4x_3$

$\hat{y} = 2 \cdot 3 \cdot (6x_1 + 4x_2 + 4x_3) = 36x_1 + 24x_2 + 24x_3$

For NN2:

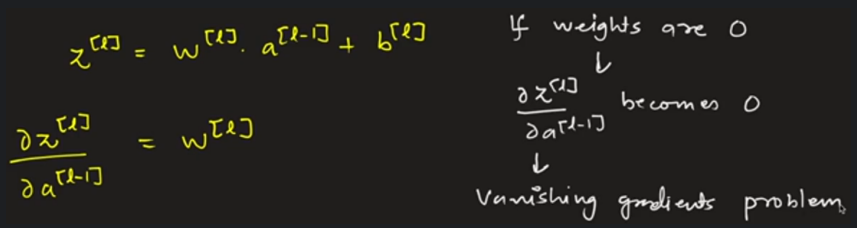
$\hat{y} = px_1 + qx_2 + rx_3$ $\quad (36, 24, 24)$

Vanishing gradients problem

- Occurs during backpropogation in deep neural networks when the gradients of the loss functions w.r.t. weights become small, approaching zero.
- This causes earlier layers (closer to the inputs) update the weights very slowly or not at all, leading to inefficient learning.

Reasons:
- Using activation functions like sigmoid and tanh.
- Repeated multiplication of small numbers during backprop reduces the gradient exponentially for earlier layers.
- In neural networks with many layers (deep), the chain rule for computing gradients leads to an exponential decrease in gradients.

Solutions:
- ReLU activation: ReLU's gradient is either 0 or 1, which avoids exponential decay.
- Batch normalization: Normalizing activation values keeps the distribution of inputs to each layer stable during training to ensure that the gradients don't shrink too much.
- Weight initialization:



Instead of zero initialization, use random, Xavier, or He initialization.

$$U\left[-\sqrt{\frac{6}{n_{in}+n_{out}}}, \sqrt{\frac{6}{n_{in}+n_{out}}}\right] \qquad \text{Normal}\left(0, \frac{2}{n_{in}}\right)$$

- Resnets (residual networks): Introduce skip connections to allow gradients to flow directly to earlier layers.

Optimizers

- Algorithm used to update weights of a NN to minimize the loss function.
- They aim to find a good local minimum of the loss by adjusting the parameters iteratively.

- Convergence speed: A good optimizer reduces the # of iterations required to reach the minimum loss.
- Stability: Prevents overshooting or getting stuck in saddle points.

Gradient descent

$$w_{new} = w_{old} - \eta \frac{\partial L}{\partial w}$$

Stochastic gradient descent

- Computes the gradients for a single data point at a time.
- Faster, but noisy updates that are sensitive to outliers and may cause oscillations.

Batch gradient descent

- Computes the gradients on the entire dataset at a time.
- Stable updates, but computationally expensive.

Mini-batch gradient descent

- Computes the gradients for a small batch of data points.

Momentum

- Accelerates GD by adding a fraction of previous update to the current update.

$$w = w + v$$
$$v = \beta v - \eta \frac{\partial L}{\partial w}$$

v: velocity; β: momentum coefficient

RMSprop

- Adjusts the learning rate for each parameter based on the moving average of square gradients.

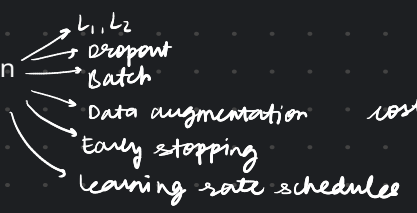
Adam

- Combines the benefits of RMSprop (dynamic learning rate) and momentum (velocity).

Hyperparameters

- External configurations or settings of an ML model that must be set before the learning process begins.
- Parameters: weights and biases
- Hyperparameters: not learned using data, but control how the training operates. E.g. learning rate, # of epochs (# of complete passes through the entire dataset), batch size, dropout rate, regularization parameters.

Regularization



$$\text{cost} = \text{error} + \lambda \sum_{i=0}^{p} |\beta_i|^2$$

regularizing term

L1 (LASSO): $$\text{loss} = \text{loss} + \lambda \sum_{i=1}^{n} |w_i|$$

L2 (Ridge): $$\text{loss} = \text{loss} + \lambda \sum_{i=1}^{n} w_i^2$$

Dropout

- Randomly drops out (sets to 0) a fraction of neurons while training.
- This prevents a few neurons to overtrain, forcing the network to learn robust patterns.
- For a dropout rate p, in a particular layer, each neuron is dropped with a probability of p.

Data augmentation

- More data prevents overfitting.
- Rotate, flip images to create more data.

Early stopping

- Using validation loss, stops the training when validation loss stops improving so that the model doesn't overfit on the training data.

Learning rate schedules

- LR is low -> convergence slow
- LR is fast -> overshoot the minima point and diverge entirely.

So we need a balance.

1. Step decay: Learning rate is reduced by a factor at regular intervals (or after a fixed # of epochs). $n_t = n_0 * \text{decay-rate}$

2. Exponential decay: The learning rate decays exponentially at regular intervals (# of epochs). $n_t = n_0 * e^{-\lambda t}$

3. Time based decay: Decreases inversely with time. $n_t = \frac{n_0}{1 + \lambda t}$